

CS224N: Natural Language Processing with Deep Learning

Lecture 2: Word Vectors Part 2, Word Senses, and Neural Classifiers

Stanford University – Spring 2024
Instructor: Christopher Manning

Contents

1	Stochastic Gradient Descent (SGD): Why It Works	3
1.1	The Surprising Benefit of Noise	3
1.2	SGD Review	3
2	Word2Vec: Deep Dive	3
2.1	The Learning Process	3
2.2	Bag of Words Model	4
2.3	Two Vectors per Word: Why?	4
2.4	Word2Vec Model Variants	5
3	Negative Sampling	5
3.1	The Computational Problem	5
3.2	Negative Sampling: The Idea	5
3.3	Negative Sampling Objective	6
3.4	Sigmoid Function	6
3.5	Sampling Strategy for Negative Examples	6
3.6	Sparse Gradient Updates	7
4	Count-Based Methods: Co-occurrence Matrices	7
4.1	Motivation	7
4.2	Building Co-occurrence Matrix	7
4.3	Example: Window-Based Co-occurrence	8
4.4	Problems with Simple Co-occurrence Vectors	8
5	Singular Value Decomposition (SVD) Methods	8
5.1	Classic Method: SVD on Co-occurrence Matrix	8
5.2	Why Raw Counts Don't Work	9
5.3	Hacks to Improve Co-occurrence Matrix	9
6	GloVe: Global Vectors	9
6.1	Key Insight: Ratios of Co-occurrence Probabilities	9
6.2	GloVe Model Formulation	10
7	Evaluating Word Vectors	10
7.1	Intrinsic vs. Extrinsic Evaluation	10
7.2	Intrinsic Evaluation Methods	11
7.2.1	Word Analogies	11
7.2.2	Word Similarity	11

7.3	Extrinsic Evaluation: Named Entity Recognition	12
8	Word Senses and Polysemy	12
8.1	The Problem of Multiple Meanings	12
8.2	Example: “Pike”	12
8.3	Approach 1: Multiple Prototypes (Huang et al. 2012)	12
8.4	Approach 2: Linear Superposition (Arora et al. 2018)	13
8.5	Standard Practice	13
9	Neural Network Classifiers	13
9.1	Named Entity Recognition (NER)	13
9.2	Window-Based Classification	14
9.3	Traditional ML vs. Neural Classification	14
9.4	Binary Neural Classifier for NER	15
9.5	Cross Entropy Loss	15
10	Neural Networks: Deep Dive	15
10.1	Neuron as Logistic Regression	15
10.2	Neural Network Architecture	16
10.3	Multi-layer Architecture	16
10.4	Matrix Notation for Neural Network Layer	16
10.5	Why Non-linearities Are Essential	17
10.6	Purpose of Intermediate Layers	17
11	Summary and Key Takeaways	17
11.1	Word2Vec Insights	17
11.2	Count-Based Methods	18
11.3	Evaluation	18
11.4	Word Senses	18
11.5	Neural Classifiers	18
12	Looking Forward	18

1 Stochastic Gradient Descent (SGD): Why It Works

1.1 The Surprising Benefit of Noise

Key Insight: Noise as a Feature

One of the interesting phenomena in neural network training is that neural networks often work **better** when you introduce noise into the system. The noise provides “jiggle” and moves things around, helping the optimization process.

Result: Stochastic Gradient Descent (SGD) is not only **way faster** but actually **works better** as an optimization method for neural networks compared to standard gradient descent.

1.2 SGD Review

Problem with standard gradient descent:

- $J(\theta)$ is a function of all windows in the corpus (potentially billions)
- Very expensive to compute full gradient
- Would wait very long before making a single update

SGD Solution:

- Repeatedly sample windows
- Update parameters after each window (or small mini-batch)
- Much faster and more effective for neural networks

2 Word2Vec: Deep Dive

2.1 The Learning Process

The Word2Vec algorithm iterates through the corpus:

1. Go through each position in the corpus
2. Using current estimates, predict probability of context words
3. Compare predictions to actual context words → compute error
4. Look at gradient and update vectors to predict surrounding words better

The Incredible Result

By doing no more than this simple prediction task, the algorithm learns word vectors that capture:

- **Semantics:** The meaning of words
- **Relationships:** Meaningful connections between different words

2.2 Bag of Words Model

Important characteristic: Word2Vec is a **bag of words model**

When computing probabilities:

- Take dot product between various possible outside words and center word
- Use these to get probability distribution over different outside words
- Compare to actual outside word in context → source of error

Key limitation:

- Doesn't know about sentence structure
- Doesn't distinguish left from right context
- Predicts the same probabilities at each position (left or right)
- Only cares about *what kind of words* appear near the center word

2.3 Two Vectors per Word: Why?

Each word has two vector representations:

- $\mathbf{v}_w$: center word vector
- $\mathbf{u}_w$: context (outside) word vector

Why two vectors?

- **Easier optimization:** Simplifies the mathematics
- In practice, we average both at the end
- They end up very close to each other anyway

Intuition for convergence:

- Consider: “The octopus has legs”
- At one position: “octopus” is center, “legs” is context
- A few time steps later: “legs” is center, “octopus” is context
- Both vectors get trained in both roles throughout the corpus

Mathematical note:

- If we make center and context vectors the same: $\mathbf{v}_w = \mathbf{u}_w$
- We get quadratic terms (x^2) at one position and linear terms (ax) at others
- This makes the math messier
- Doesn't improve performance—just a mathematical convenience

2.4 Word2Vec Model Variants

Two main architectures:

1. **Skip-gram (SG)**: Predict context words given center word
 - This is what we've been studying
 - Position-independent predictions
2. **Continuous Bag of Words (CBOW)**: Predict center word from context words
 - Uses bag of context words as input

Three loss function options:

1. **Naïve softmax**: Simple but computationally expensive (many output classes)
2. **Hierarchical softmax**: More optimized variant
3. **Negative sampling**: Efficient approximation (discussed below)

3 Negative Sampling

3.1 The Computational Problem

The standard softmax is expensive:

$$P(o|c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in V} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad (1)$$

Why expensive?

- Denominator sums over *entire vocabulary* (e.g., 400,000 words)
- Each term requires dot product between 100–300 dimensional vectors
- Enormous amount of computation per update

Note: With modern computers and high computational capability, softmax is less problematic than before. However, negative sampling remains a popular and efficient alternative.

3.2 Negative Sampling: The Idea

Core concept: Train binary logistic regressions instead of multi-class softmax

- Create **true pairs**: (center word, actual context word) → should have high probability
- Create **noise pairs**: (center word, random word) → should have low probability

3.3 Negative Sampling Objective

Take K negative samples and minimize:

$$J_{\text{neg-sample}}(\mathbf{u}_o, \mathbf{v}_c, U) = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{k \in \{\text{K neg samples}\}} \log \sigma(-\mathbf{u}_k^\top \mathbf{v}_c) \quad (2)$$

Components:

- **First term:** $-\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c)$
 - Real outside word
 - Want this to have high probability (near 1)
 - Negated because we're minimizing
- **Second term:** $-\sum_k \log \sigma(-\mathbf{u}_k^\top \mathbf{v}_c)$
 - Sampled negative words
 - Want their probability to be small (near 0)
 - Note the negative sign inside σ

3.4 Sigmoid Function

Using **sigmoid** instead of softmax:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

Properties:

- Symmetric function
- For real outside word: want $\mathbf{u}_o^\top \mathbf{v}_c$ large $\rightarrow \sigma \approx 1$
- For negative samples: want probability near 0
- The negative sign works because sigmoid is symmetric

3.5 Sampling Strategy for Negative Examples

How to choose negative samples?

Not uniform random sampling! Instead, use weighted sampling:

$$P(w) = \frac{U(w)^{3/4}}{Z} \quad (4)$$

where:

- $U(w)$ is the **unigram distribution** (word frequency)
- Power of 3/4 increases sampling probability of less frequent words
- Z is normalization constant

Why this strategy?

- Common words like “the” appear very frequently (about 10% of corpus)
- Raising to power 3/4 moves somewhat toward uniform sampling

- Less frequent words get sampled more than their raw frequency
- Better results than pure frequency-based or uniform sampling
- Power of 0 would give uniform; power of 1 gives exact frequencies

How many negative samples?

- Typically $K = 5$ to 10 negative samples per true pair
- Much less than vocabulary size (e.g., 400,000)

3.6 Sparse Gradient Updates

Computational Efficiency

In each window with negative sampling:

- At most $2m + 1$ words (window size m)
- Plus $2km$ negative words
- Gradient $\nabla_{\theta} J_t(\theta)$ is **very sparse**

Implementation considerations:

- Only update word vectors that actually appear
- Need sparse matrix update operations
- Or maintain hash table for word vectors
- Important for distributed computing: avoid sending gigantic updates

4 Count-Based Methods: Co-occurrence Matrices

4.1 Motivation

Question: There's something unusual about iterating through the entire corpus (perhaps many times). Why don't we just accumulate statistics of what words appear near each other?

4.2 Building Co-occurrence Matrix

Two main approaches:

1. **Window-based:** Similar to word2vec
 - Use window around each word
 - Captures syntactic and semantic information
 - Creates "word space"
2. **Document-based:** Full document co-occurrence
 - Gives general topics
 - All sports terms have similar entries
 - Leads to "Latent Semantic Analysis" (LSA)
 - Creates "document space"

4.3 Example: Window-Based Co-occurrence

Corpus:

- I like deep learning
- I like NLP
- I enjoy flying

Co-occurrence matrix (window length 1, symmetric):

	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

4.4 Problems with Simple Co-occurrence Vectors

- Vectors increase in size with vocabulary
- **Very high dimensional:** Requires lots of storage (though sparse)
- **Sparsity issues:** Subsequent classification models less robust

Solution: Use **low-dimensional dense vectors**

- Store “most” important information in fixed, small number of dimensions
- Usually 25–1000 dimensions (similar to word2vec)
- Method: Dimensionality reduction

5 Singular Value Decomposition (SVD) Methods

5.1 Classic Method: SVD on Co-occurrence Matrix

Approach: Apply SVD to co-occurrence matrix X :

$$X = U\Sigma V^T \tag{5}$$

where:

- U and V are orthonormal matrices (unit vectors, mutually orthogonal)
- Σ is diagonal matrix of singular values

Dimensionality reduction:

- Retain only k largest singular values
- $\hat{X}$ is best rank- k approximation to X (least squares sense)

- Classic linear algebra result

Problem: Matrix size is $|V| \times |V|$ (e.g., $400,000 \times 400,000$)

- Much worse than word vectors ($|V| \times d$ where $d \approx 100$)
- Expensive to compute for large matrices

5.2 Why Raw Counts Don't Work

Critical Issue

Running SVD on raw counts doesn't work well!

Main problem: Function words (the, he, has) are too frequent

- Syntax dominates the representation
- Overwhelms semantic information

5.3 Hacks to Improve Co-occurrence Matrix

Multiple fixes explored (Rohde et al. 2005 in COALS model):

1. **Log frequencies:** $\log(\text{count})$
2. **Capping:** $\min(X, t)$ where $t \approx 100$
3. **Ignore function words:** Remove high-frequency function words
4. **Ramped windows:** Count closer words more than farther words
5. **Pearson correlations:** Use correlations instead of counts, set negative values to 0

Result with hacks:

- Interesting semantic patterns emerge
- Example: Doer-of-event becomes linear meaning component
- Methods *sort of* worked, but never worked very well
- Never as effective as modern neural methods

6 GloVe: Global Vectors

6.1 Key Insight: Ratios of Co-occurrence Probabilities

Goal: Encode meaning components as linear operations in vector space

Central Idea

Ratios of co-occurrence probabilities can encode meaning components better than raw probabilities.

Example with ice/steam:

Consider words related to different states of matter:

Observations:

	$P(\cdot \text{ice})$	$P(\cdot \text{steam})$	Ratio	Interpretation
solid	1.9×10^{-4}	2.2×10^{-5}	8.9	large
gas	6.6×10^{-5}	7.8×10^{-4}	8.5×10^{-2}	small
water	2.2×10^{-3}	3.0×10^{-3}	0.96	≈ 1
fashion	1.7×10^{-5}	1.8×10^{-5}	1.36	≈ 1

- For “solid”: ratio is large (ice-related)
- For “gas”: ratio is small (steam-related)
- For “water”: ratio ≈ 1 (related to both)
- For “fashion”: ratio ≈ 1 (unrelated to either)

Key insight: The ratio captures a **direction in the space** corresponding to the solid-liquid-gas dimension of physics!

6.2 GloVe Model Formulation

Question: How to capture ratios as linear meaning components?

Answer: Use log-bilinear model with vector differences

Strategy:

- Convert division into subtraction using logarithm
- Model: $\mathbf{w}_i^\top \mathbf{w}_j \approx \log P(j|i) = \log X_{ij} - \log X_i$

GloVe objective function:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (6)$$

where:

- X_{ij} : co-occurrence count of words i and j
- $f(X_{ij})$: weighting function (downweights rare and very frequent co-occurrences)
- $b_i, \tilde{b}_j$: bias terms

Properties:

- **Fast training**
- **Scalable** to huge corpora
- Combines benefits of count-based and prediction-based methods

7 Evaluating Word Vectors

7.1 Intrinsic vs. Extrinsic Evaluation

Definition 1 (Intrinsic Evaluation). *Evaluation on a specific/intermediate subtask*

Advantages:

- Fast to compute

- Helps understand the system

Disadvantages:

- Distant from downstream task
- Not clear if helpful unless correlation to real task is established

Definition 2 (Extrinsic Evaluation). *Evaluation on a real task (e.g., question answering, summarization)*

Advantages:

- Directly measures usefulness
- If replacing one subsystem improves accuracy $\rightarrow$ winning!

Disadvantages:

- Can take long time to compute
- Unclear if subsystem is the problem or its interaction with other components

7.2 Intrinsic Evaluation Methods

7.2.1 Word Analogies

Task format: $a : b :: c : ?$

Example: man:woman :: king:?

Method:

$$\text{answer} = \arg \max_w \text{cosine}(\mathbf{w}, \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}}) \quad (7)$$

Important: Discard input words from search!

Problem: What if information is there but not linear?

7.2.2 Word Similarity

Task: Correlation with human judgments of similarity

Example dataset: WordSim353

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10.00
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Evaluation metric: Spearman rank correlation between:

- Human similarity judgments
- Model similarity scores (cosine distance)

7.3 Extrinsic Evaluation: Named Entity Recognition

Task: Identify and classify named entities

Example:

Chris Manning lives in Palo Alto.
[PER] [PER] [LOC] [LOC]

Evaluation: F1 score on NER task

Typical results: Good word vectors improve NER performance by several percentage points compared to baseline without word vectors.

8 Word Senses and Polysemy

8.1 The Problem of Multiple Meanings

Most words have lots of meanings!

- Especially common words
- Especially words that have existed for a long time

8.2 Example: “Pike”

The word “pike” has many meanings:

1. A sharp point or staff
2. A type of elongated fish
3. A railroad line or system
4. A type of road
5. The future (“coming down the pike”)
6. A type of body position (as in diving)
7. To kill or pierce with a pike (verb)
8. To make one’s way (“pike along”) (verb)
9. In Australian English: to pull out from doing something
 - “I reckon he could have climbed that cliff, but he piked!”

Question: Does one vector capture all these meanings, or do we have a mess?

8.3 Approach 1: Multiple Prototypes (Huang et al. 2012)

Method:

1. Cluster word windows around words
2. Assign each word token to a cluster
3. Learn separate vectors: bank₁, bank₂, approach₁, approach₂, approach₃, etc.

8.4 Approach 2: Linear Superposition (Arora et al. 2018)

Key Result

Different senses of a word reside in a **linear superposition** (weighted sum) in standard word embeddings.

Mathematical formulation:

$$\mathbf{v}_{\text{pike}} = \alpha_1 \mathbf{v}_{\text{pike}_1} + \alpha_2 \mathbf{v}_{\text{pike}_2} + \alpha_3 \mathbf{v}_{\text{pike}_3} \quad (8)$$

where:

$$\alpha_i = \frac{f_i}{f_1 + f_2 + f_3} \quad (9)$$

for frequency f_i of sense i .

Surprising result:

- Using ideas from sparse coding, you can actually **separate out the senses**
- Works when senses are relatively common

8.5 Standard Practice

What's usually done:

- Only one vector per word (e.g., one vector for “jaguar”)
- This vector is a **weighted average** of the sense-specific vectors
- Weights proportional to sense frequencies

Why this works reasonably well:

- Context disambiguates in downstream tasks
- Modern contextual embeddings (BERT, GPT) generate different vectors based on context

9 Neural Network Classifiers

9.1 Named Entity Recognition (NER)

Task: Find and classify names in text by labeling word tokens

Examples:

- Last night, Paris Hilton wowed in a sequin gown.
 - Paris: PER, Hilton: PER
- Samuel Quinn was arrested in the Hilton Hotel in Paris in April 1989.
 - Samuel: PER, Quinn: PER
 - Hilton: LOC, Hotel: LOC
 - Paris: LOC, April: DATE, 1989: DATE

Applications:

- Tracking mentions of entities in documents
- Question answering (answers often are named entities)
- Sentiment analysis related to specific entities
- Entity linking into knowledge bases (e.g., Wikidata)

9.2 Window-Based Classification

Approach: Classify each word in its context window

Method:

1. Create window of neighboring words around center word
2. Concatenate word vectors in window
3. Train classifier on this representation

Example: Classify “Paris” as location with window size 2:

the museums in Paris are amazing

$$\mathbf{x}_{\text{window}} = [\mathbf{x}_{\text{museums}} \ \mathbf{x}_{\text{in}} \ \mathbf{x}_{\text{Paris}} \ \mathbf{x}_{\text{are}} \ \mathbf{x}_{\text{amazing}}]^T \in \mathbb{R}^{5d} \quad (10)$$

Run classifier for each class on vector centered on each word in sentence.

9.3 Traditional ML vs. Neural Classification

Traditional ML/Stats approach:

- Learn only classifier parameters W
- Input representation x has sparse symbolic features (fixed)
- Linear decision boundary

Neural network approach differs:

- Learn *both* W and distributed word representations
- Word vectors re-represent one-hot vectors
- Move words around in intermediate layer vector space
- Use deep networks with multiple layers
- Creates **non-linear classifier** in terms of original input space

Key Understanding

In neural networks:

- Word vectors represent actual words as symbols
- Can move them around in space to create non-linear decision boundaries
- **Final classifier is linear in the learned representation**
- **But non-linear in the original input space**

Conceptually:

- We have an embedding layer: $\mathbf{x} = L\mathbf{e}$ (where $\mathbf{e}$ is one-hot)
- Multiple hidden layers re-represent and compose data
- Final layer uses linear classifier on learned representations

9.4 Binary Neural Classifier for NER

Architecture for classifying if center word is a location:

$$\mathbf{x} = [\mathbf{x}_{\text{museums}} \ \mathbf{x}_{\text{in}} \ \mathbf{x}_{\text{Paris}} \ \mathbf{x}_{\text{are}} \ \mathbf{x}_{\text{amazing}}]^T \quad (11)$$

$$s = \mathbf{u}^T h, \quad \text{where } h = f(W\mathbf{x} + \mathbf{b}) \quad (12)$$

$$\hat{y} = \sigma(s) = \frac{1}{1 + e^{-s}} \quad (13)$$

where:

- f : element-wise non-linear function (logistic, tanh, ReLU)
- $\hat{y}$: predicted probability of class (location)

9.5 Cross Entropy Loss

Previous formulation:

- Maximize probability of correct class y
- Equivalently: minimize negative log probability

Information theory perspective:

Definition 3 (Cross Entropy). *Let p be true probability distribution and q be model probability distribution. The cross entropy is:*

$$H(p, q) = - \sum_{c=1}^C p(c) \log q(c) \quad (14)$$

For classification:

- True distribution: $p = [0, \dots, 0, 1, 0, \dots, 0]$ (one-hot)
- Only term that survives: $-\log p(y_i | x_i)$
- This is our loss function

Implementation: Use `torch.nn.CrossEntropyLoss()` in PyTorch

10 Neural Networks: Deep Dive

10.1 Neuron as Logistic Regression

A single neuron performs logistic regression:

$$h_{w,b}(\mathbf{x}) = f(\mathbf{w}^T \mathbf{x} + b) \quad (15)$$

where:

- $f(z) = \frac{1}{1+e^{-z}}$ (activation function)
- $\mathbf{w}, b$ are parameters (weights and bias)
- b can be thought of as:
 - An “always on” bias feature (class prior)
 - Or separated out as explicit bias term

10.2 Neural Network Architecture

Core Idea

A neural network is running several logistic regressions at the same time, then feeding outputs into more logistic regressions.

Key insight:

- We don't decide ahead of time what intermediate regressions predict
- The final loss function directs what intermediate hidden variables should be
- Learn representations that make it easier to classify downstream

10.3 Multi-layer Architecture

Process:

1. Feed input vector through multiple logistic regression functions $\rightarrow$ vector of outputs
2. Feed these outputs into another layer of logistic regressions
3. Compose functions multiple times
4. Final layer produces prediction

Result: This allows us to:

- Re-represent and compose data multiple times
- Learn classifier that is **highly non-linear** in original inputs
- But typically **linear in pre-final layer representations**

10.4 Matrix Notation for Neural Network Layer

Individual neuron equations:

$$a_1 = f(W_{11}x_1 + W_{12}x_2 + W_{13}x_3 + b_1) \quad (16)$$

$$a_2 = f(W_{21}x_1 + W_{22}x_2 + W_{23}x_3 + b_2) \quad (17)$$

$$\vdots \quad (18)$$

Matrix form:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad (19)$$

$$\mathbf{a} = f(\mathbf{z}) \quad (20)$$

where activation f is applied **element-wise**:

$$f([z_1, z_2, z_3]) = [f(z_1), f(z_2), f(z_3)] \quad (21)$$

10.5 Why Non-linearities Are Essential

Theorem 1 (Need for Non-linearity). *Without non-linearities, deep neural networks cannot do anything more than a linear transform.*

Proof sketch:

- Without non-linearities: $h = W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x} = W\mathbf{x}$
- Extra layers just compile down to single linear transform
- No benefit from depth

With non-linearities:

- Can approximate arbitrarily complex functions
- Each layer learns progressively more abstract representations
- Depth provides exponential increase in representational power

10.6 Purpose of Intermediate Layers

Key Function

Intermediate layers learn to represent input data in ways that make it:

- Easier to classify
- Easier to interpret
- More useful for downstream tasks in the network

Example in NER:

- Input layer: raw word vectors
- Hidden layer 1: might learn syntactic patterns
- Hidden layer 2: might learn semantic entity patterns
- Output layer: classification (is it a location?)

11 Summary and Key Takeaways

11.1 Word2Vec Insights

1. **SGD with noise is beneficial:** Helps neural network optimization
2. **Bag of words model:** Word2Vec doesn't consider word order or structure
3. **Two vectors per word:** Easier optimization; converge to similar values
4. **Negative sampling:** Efficient alternative to softmax
 - Sample 5–10 negative examples
 - Use unigram distribution raised to power $3/4$

11.2 Count-Based Methods

1. **Co-occurrence matrices:** Capture word statistics directly
2. **SVD methods:** Classic dimensionality reduction
 - Raw counts don't work
 - Need various hacks (log, capping, etc.)
 - Never worked as well as neural methods
3. **GloVe:** Combines count-based and prediction-based approaches
 - Uses ratios of co-occurrence probabilities
 - Fast and scalable

11.3 Evaluation

1. **Intrinsic:** Fast, specific tasks (analogies, similarity)
2. **Extrinsic:** Real tasks (NER, QA) – what really matters

11.4 Word Senses

1. **Multiple meanings:** Most words are polysemous
2. **Standard practice:** One vector = weighted average of senses
3. **Linear superposition:** Senses can be separated (Arora et al. 2018)
4. **Future:** Contextual embeddings (BERT, GPT) provide context-specific representations

11.5 Neural Classifiers

1. **Learn representations:** Not just classifier weights
2. **Non-linear in input space:** Linear in learned representation space
3. **Deep networks:** Multiple layers compose representations
4. **Non-linearities essential:** Enable learning complex functions
5. **Intermediate layers:** Learn progressively more abstract features

12 Looking Forward

The concepts from these first two lectures form the foundation for modern NLP:

- **Word embeddings:** Still used, though now contextual (BERT, GPT)
- **Neural architectures:** Extended to recurrent networks, transformers
- **Optimization:** SGD principles apply throughout deep learning
- **Evaluation:** Intrinsic vs. extrinsic remains key distinction

Next steps in the course:

- Dependency parsing

- Recurrent neural networks
- Attention mechanisms
- Transformer architectures
- Pre-training and fine-tuning (BERT, GPT)
- Language model agents